

Configuring WebSphere MQ and Liberty profile for SSL connectivity

Introduction

To connect to WebSphere WMQ (WMQ) over SSL, the queue manager must use a certificate for the client to securely connect to the queue. You can create self-signed certificates or request a certificate from the certification authority (CA) or in-house certification authority.

Self-signed certificates are commonly used in a test environment. The steps or commands provided in this document are specific to the following versions:

- Java, version 1.7 SR3 -FP10
- WMQ, version 8.0.0.2
- Liberty profile, version 8.5.5.6
- OS, version RHEL 6.6

The instructions might vary from one version to another version. For more information about the instructions specific to a version, see the corresponding documentation.

The implementation of SSL server authentication requires the configuration of SSL on WMQ and Liberty profile.

Configuring SSL on WMQ

Perform the following steps to configure SSL on WMQ:

1. Create a key repository for the queue manager.
2. Create a self-signed certificate for the queue manager.
3. Extract the certification authority (CA) certificate.
4. Add the keystore that you created in step 1 to the WMQ queue manager.
5. Configure the required Cipher specification at the WMQ channel level.

Creating a key repository for the queue manager

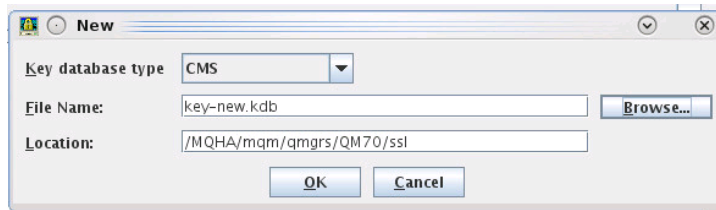
To create a key repository for the queue manager, perform the following steps:

1. To launch the IBM Key Management (strmqikm) GUI, you can either use the console or use any xWin display redirection method.
Note: In a typical WMQ installation, the IBM Key Management program (strmqikm) can be found under **/opt/mqm/bin** folder. Ensure that you run this program as a root user.
2. Navigate to **Key Database File > New**.
3. Create a key repository for the queue manager as follows:
 - From the **Key database type** drop-down list, select CMS.
 - In the **File Name** field, enter the file name of the key database, for example **key_new.kdb**.

- In the **Location** field, enter the folder where you want to store the key database file. For example, /MQHA/mqm/qmgrs/QM70/ssl.

Note: It is recommended that the location where you store this key database or repository is same as the location where you store the SSL key repository for your WMQ queue manager keystore.

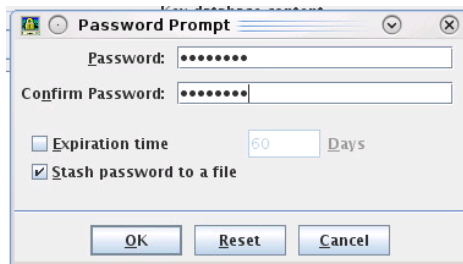
4. Click **OK**.



5. Enter the password for the key database, for example, **key_new.kdb file**.

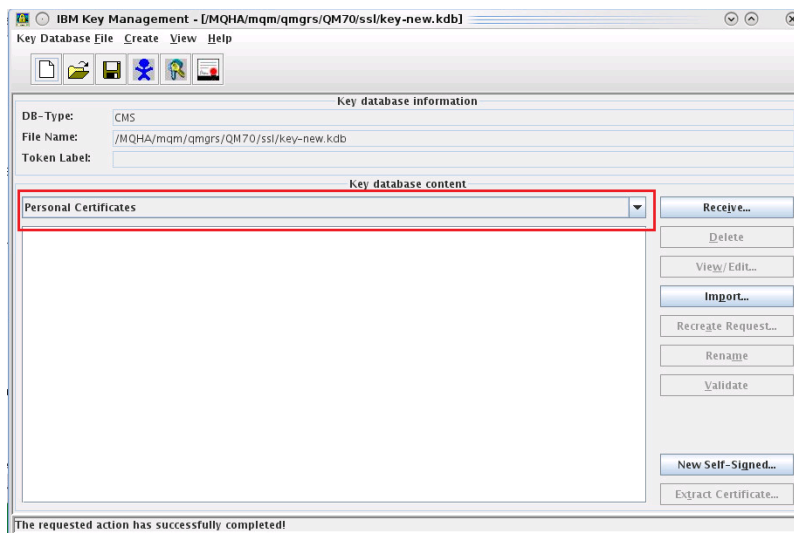
6. Select the **Stash password to a file** check box.

7. Click **OK**. The key repository for the queue manager, **QM70** is created.



After creating the key repository, the installed certification authority certificate that are provided with the key manager are displayed as shown in the following screenshot.

From the **Personal Certificates** drop-down list, select Personal Certificate and proceed with the creation of self-signed certification. For more information, see section, [Creating a self-signed certificate](#).



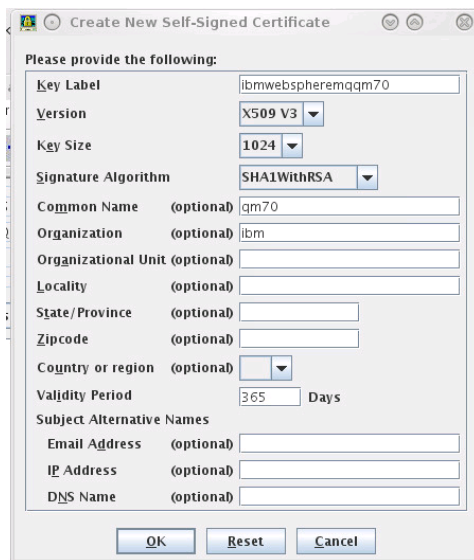
Creating a self-signed certificate

To complete the SSL configuration between the queue manager and client programs, create a self-signed certificate in the same key repository, for example, **key_new.kdb**.

To create a self-signed certificate:

1. Click the **New Self-Signed Certificate** button.
2. In the **Key Label** field, enter the key label, for example **ibmwebspheremqm70**.
Important: Ensure that the key label is prefixed with `ibmwebspheremq` followed by the queue manager name in lowercase, which is `qm70`.
3. In the **Common Name** field, enter the common name, for example **qm70**.
4. In the **Organization** field, enter the company name, for example **IBM**.
5. Click **OK**.

Important: The remaining fields are optional and can be left blank.



The screenshot shows a Windows-style dialog box titled "Create New Self-Signed Certificate". It contains a group box "Please provide the following:" with several input fields and dropdown menus. The "Key Label" field is filled with "ibmwebspheremqm70". The "Version" dropdown is set to "X509 V3". The "Key Size" dropdown is set to "1024". The "Signature Algorithm" dropdown is set to "SHA1WithRSA". The "Common Name" field is filled with "qm70" and is marked as optional. The "Organization" field is filled with "ibm" and is marked as optional. The "Organizational Unit", "Locality", "State/Province", "Zipcode", "Country or region", "Email Address", "IP Address", and "DNS Name" fields are all empty and marked as optional. The "Validity Period" is set to "365 Days". At the bottom of the dialog are three buttons: "OK", "Reset", and "Cancel".

The certificate is displayed with an asterisk to the left of the name (Asterisk indicates that this is the default certificate for the key repository).

Now, the queue manager has a certificate. The queue manager presents this certificate to the WMQ client, when the client connects.

To validate the queue manager's certificate, the client needs the certification authority (CA) certificate in its local keystore or as part of the Java runtime environment trust store, **cacert**.

Extract the Certification authority (CA) certificate

To extract the CA certificate, perform the following steps:

1. Click **Extract Certificate**, which is at the bottom right-hand corner.
2. In the **File Name** field, enter the name of the certificate, for example, key_new.arm.
3. In the **Location** field, enter the location where the certificate is stored, for example, /MQHA/mqm/qmgrs/QM70/ssl.
4. Click **OK**. The **key_new.arm** file is created and copied to the **/MQHA/mqm/qmgrs/QM70/ssl** folder.
5. Transfer the CA Certificate file(**key_new.arm**) to the client's machine where we import this certificate to client side key repository. Before that, close the key repository with which you were working by clicking **Close** from the menu.

Configuring the WMQ queue manager keystore

You can configure the queue manager keystore by using the IBM WMQ explorer. If you want to configure the queue manager keystore by using commands, refer to the *WMQ documentation*.

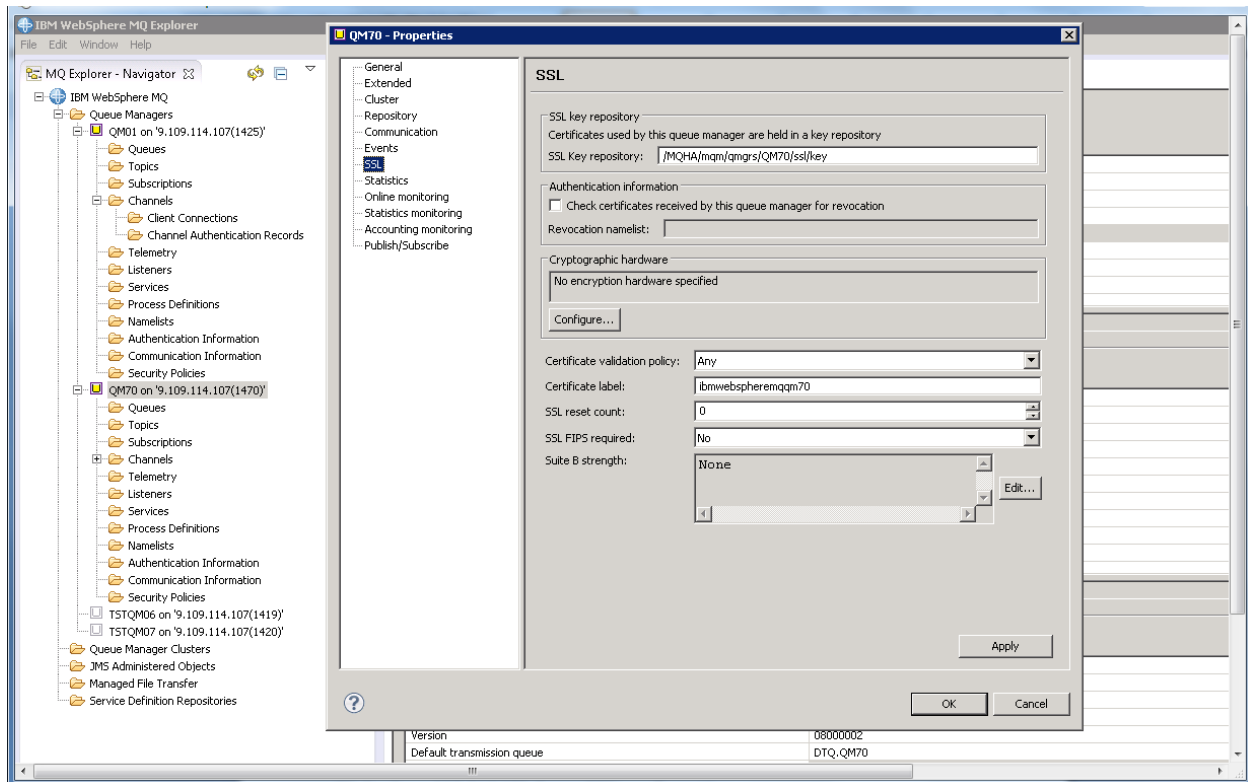
To configure WMQ queue manager keystore by using the IBM WMQ explorer, perform the following steps:

1. Navigate to the Queue Manager Properties screen. For this, right-click the menu option of Queue Manager → Properties.
2. In the **SSL Key repository**, enter the path of the SSL key repository along with the key repository name, for example, /MQHA/mqm/qmgrs/QM70/ssl/key_new (the key repository name with no extension).

Note: It is recommended that the location where you store the SSL key repository is same as the location where the key repository for the queue manager is created (see section, [Creating a key repository for the queue manager](#)). However, if the path is different, copy all the contents to this location.

3. In the **Certificate label** field, enter the certificate label, for example, ibmwebsphermqm70. This is same as the key label provided in the certificate.
4. Click **OK**.

Important: The remaining fields are optional and can be left blank.



Configuring WMQ SSL channel

To enable queue channels to communicate over SSL, at the WMQ side, perform the following steps:

1. Navigate to the **Channel's Properties** screen by clicking **Channels**. The list of channels is displayed on the right-hand side.
2. Right-click your channel from the list, for example **Channel (QM70.SSL) > Properties**.
3. In the **SSL Cipher Spec** field, enter TLS_RSA_WITH_NULL_SHA256.

Note: Ensure that the Cipher Specification that you configure at the channel level and the Cipher Suite at the Liberty side are compatible. All the TLS Ciphers are present under **TLS* Cipher spec**. For more information about Cipher Specification and Cipher Suite mapping documentation, go to: [http://www-01.ibm.com/support/knowledgecenter/SSFKSJ_7.5.0/com.ibm.mq.sec.doc/q014260 .htm](http://www-01.ibm.com/support/knowledgecenter/SSFKSJ_7.5.0/com.ibm.mq.sec.doc/q014260.htm). You can also search for **CipherSpec** in WMQ Knowledge Center.

Here, TLS 1.2 cipher **TLS_RSA_WITH_NULL_SHA256** is used as Cipher Spec and the corresponding Cipher Suite is **SSL_RSA_WITH_NULL_SHA256**.

4. In the **SSL Authentication** field, enter optional.

IBM WebSphere MQ Explorer

MQ Explorer - Navigator

- IBM WebSphere MQ
 - Queue Managers
 - QM01 on '9.109.114.107'(1425)
 - Queues
 - Topics
 - Subscriptions
 - Channels
 - Client Connections
 - Channel Authentication Records
 - Telemetry
 - Listeners
 - Services
 - Process Definitions
 - Nameslists
 - Authentication Information
 - Communication Information
 - Security Policies
 - QM70 on '9.109.114.107'(1470)
 - Queues
 - Topics
 - Subscriptions
 - Channels
 - Client Connections
 - Channel Authentication Records
 - Telemetry
 - Listeners
 - Services
 - Process Definitions
 - Nameslists
 - Authentication Information
 - Communication Information
 - Security Policies
 - TSTQM06 on '9.109.114.107'(1419)
 - TSTQM07 on '9.109.114.107'(1420)
 - Queue Manager Clusters
 - JMS Administered Objects
 - Managed File Transfer
 - Service Definition Repositories

MQ Explorer - Content

Channels

Filter: Standard for Channels

Channel name	Channel type	Overall channel status	Conn name	Transmission queue	MCA user ID	Batch size	Disconnect interval	He
QM70.NONSSL	Server-connection	Inactive				0	30	
QM70.SSL	Server-connection	Running				0	30	
SYSTEM.ADMIN.SVRCONN	Server-connection	Running				0	30	
SYSTEM.AUTO.RECEIVER	Receiver	Inactive			50		30	
SYSTEM.AUTO.SVRCONN	Server-connection	Inactive				0	30	
SYSTEM.DEF.CLUSRCVR	Cluster-receiver	Inactive			50	6000	30	
SYSTEM.DEF.CLUSDR	Cluster-sender	Inactive			50	6000	30	
SYSTEM.DEF.RECEIVER	Receiver	Inactive			50		30	
SYSTEM.DEF.REQUESTER	Requester	Inactive			50		30	
SYSTEM.DEF.SENDER	Sender	Inactive			50	6000	30	
SYSTEM.DEF.SERVER	Server	Inactive			50	6000	30	
SYSTEM.DEF.SVRCONN	Server-connection	Inactive				0	30	

IBM WebSphere MQ Explorer

MQ Explorer - Navigator

- IBM WebSphere MQ
 - Queue Managers
 - QM01 on '9.109.114.107'(1425)
 - Queues
 - Topics
 - Subscriptions
 - Channels
 - Client Connections
 - Channel Authentication Records
 - Telemetry
 - Listeners
 - Services
 - Process Definitions
 - Nameslists
 - Authentication Information
 - Communication Information
 - Security Policies
 - QM70 on '9.109.114.107'(1470)
 - Queues
 - Topics
 - Subscriptions
 - Channels
 - Client Connections
 - Channel Authentication Records
 - Telemetry
 - Listeners
 - Services
 - Process Definitions
 - Nameslists
 - Authentication Information
 - Communication Information
 - Security Policies
 - TSTQM06 on '9.109.114.107'(1419)
 - TSTQM07 on '9.109.114.107'(1420)
 - Queue Manager Clusters
 - JMS Administered Objects
 - Managed File Transfer
 - Service Definition Repositories

QM70.SSL - Properties

General

SSL

CipherSpec

Set message security for this end of the channel

SSL Cipher Spec: TLS_RSA_WITH_NULL_SHA256

(WARNING Weak CipherSpec) TLS 1.2, 256-bit Secure Hash Algorithm, no encryption

☐ Accept only certificates with Distinguished Names matching these values:

SSL Authentication: Optional

Certificate label:

Apply

OK Cancel

Configuring Liberty WMQ SSL

Perform the following steps to configure SSL between Liberty and WMQ:

1. Import the CA certificate that you extracted in step 3 of section [Configuring WMQ SSL channel](#).
2. Configuring the Liberty profile by adding the keystore and the required Cipher suite in the `server.xml` file.

Importing the CA part in the client's key repository

The installation of CA part in the clients' key repository comprises of the following steps that are executed on the client machine.

1. Create a key repository(`localKeyStore.jks`) for Liberty if it is not created already
2. Copy or transfer the CA (`key_new.arm`) file to the client machine.
3. Import the CA certificate (`key_new.arm`) to the client side keystore (`localKeyStore.jks`) that is configured for Liberty by running the following keystore command.

```
keytool -import -trustcacerts -alias ibmwebspheremqqm70 -file key_new.arm -  
keystore localKeyStore.jks -storepass password -storetype jks
```

Here,

- **alias** refers to `ibmwebspheremqqm70`, which is same as the key label provided in the certificate.
- **file** refers to `key_new.arm` that is extracted CA certificate from the queue manager key repository.
- **keystore** refers to `localKeyStore.jks`, which is the Liberty keystore.
- **storepass** refers to `password`, which is the Liberty keystore password.

Liberty profile configuration

To restrict the SSL communication cipher that is used, in the **server.xml** file, specify the cipher in the **sslCipherSuite** attribute of the **wmqJms** channel property as follows:

```
<jmsConnectionFactory connectionManagerRef="MDRConnectionMgr"  
jndiName="jms/PrvQueueCF" containerAuthDataRef="auth1">  
  <properties.wmqJms channel="QM70.SSL"  
connectionNameList="198.109.114.107(1470),198.109.114.134(1470)"  
queueManager="QM70" transportType="CLIENT"  
sslCipherSuite="SSL_RSA_WITH_NULL_SHA256" userName="user123"  
password="pass123"/>  
</jmsConnectionFactory>
```

With the latest version of Java, for a self signed certificate, you might encounter the following error:

HTTP transport error ... Certificate Chaining Error

This is because, the self signed certificate is not signed by any trusted third-party certification authority (CA).

Workarounds include:

- If you have access to Java runtime, import the CA certificate **key_new.arm** to cacerts Java truststore that is present under **\$JAVA_HOME/jre/lib/security** folder.
- Alternatively, you can perform the following steps:
 1. Create a local truststore.
 2. Import the CA certificate, **key_new.arm**. For the command to import a CA certificate to the existing keystore see section, [*Importing the CA part in the client's key repository*](#).
 3. Point your application to the local truststore. For this, in the Liberty runtime, add the following line as the path to the truststore in the **jvm.options** file that is present under the server directory.

Djavax.net.ssl.trustStore=<LOCAL_TRUSTSTORE_LOC>/localTrustStore.jks

The advantage with this approach is that the truststore is changed at the JVM level without changing the default java security.